O-RAN.WG10.TS.OnbdSMOSGAP-R004-v02.00

Technical Specification

O-RAN Work Group 10 (OAM for O-RAN)

Onboarding SMOS General Aspects and Principles

O-RAN ALLIANCE e.V., Buschkauler Weg 27, 53347 Alfter, Germany

Register of Associations, Bonn VR 11238, VAT ID DE321720189

Contents

List of figures

List of tables

Foreword

This Technical Specification (TS) has been produced by WG10 of the O-RAN Alliance.

The content of the present document is subject to continuing work within O-RAN and may change following formal O-RAN approval. Should the O-RAN Alliance modify the contents of the present document, it will be re-released by O-RAN with an identifying change of version date and an increase in version number as follows:

version xx.yy.zz

where:

- xx: the first digit-group is incremented for all changes of substance, i.e. technical enhancements, corrections, updates, etc. (the initial approved document will have xx=01). Always 2 digits with leading zero if needed.
- yy: the second digit-group is incremented when editorial only changes have been incorporated in the document. Always 2 digits with leading zero if needed.
- zz: the third digit-group included only in working versions of the document indicating incremental changes during the editing process. External versions never include the third digit-group. Always 2 digits with leading zero if needed.

Modal verbs terminology

In the present document "**shall**", "**shall not**", "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the O-RAN Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in O-RAN deliverables except when used in direct citation.

1 Scope

The present document specifies the general aspects and principles of the Onboarding SMOS as described in the SMO Decomposition TR [i.3], clause 5.11. The specification identifies where the service fits into the Application Lifecycle as described in the OAM Architecture [i.5], clause 6. Beyond the general aspects and principles, this document provides the use cases from the user perspective and specifies the service procedures, requirements, and information model for the service.

2 References

2.1 Normative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies. In the case of a reference to a 3GPP document (including a GSM document), a non-specific reference implicitly refers to the latest version of that document in 3GPP Release 18.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, O-RAN cannot guarantee their long-term validity.

The following referenced documents are necessary for the application of the present document.

[1] O-RAN.WG11-O-RAN-Security-Requirements-Specification: “O-RAN Security Requirements”

[2] ETSI GS NFV-SOL 004 v5.1.1, " Network Functions Virtualisation (NFV) Release 5; Protocols and Data Models; VNF Package and PNFD Archive specification", July 2024

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies. In the case of a reference to a 3GPP document (including a GSM document), a non-specific reference implicitly refers to the latest version of that document in 3GPP Release 18.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, O-RAN cannot guarantee their long-term validity.

The following referenced documents are not necessary for the application of the present document, but they assist the user with regard to a particular subject area.

[i.1] 3GPP TR 21.905: “Vocabulary for 3GPP Specifications”.

[i.2] O-RAN WG1-O-RAN Architecture Description: “O-RAN Architecture Description”

[i.3] O-RAN WG1-Decoupled-SMO-Architecture: “Decoupled SMO Architecture”

[i.4] O-RAN.WG6.ORC, “Cloudification and Orchestration Use Cases for O-RAN Virtualized RAN”

[i.5] O-RAN.WG10.OAM-Architecture: "OAM Architecture"

[i.6] Rec.ITU-T M.3020: “Management interface specification methodology “.

3 Definition of terms, symbols and abbreviations

3.1 Terms

For the purposes of the present document, the terms given in 3GPP TR 21.905 [i.1] apply.

NOTE: A term defined in the present document takes precedence over the definition of the same term, if any, in 3GPP TR 21.905 [i.1].

SMO: A Service Management and Orchestration framework as described in the O-RAN WG1.OAD [i.2].

Application: The software aspect of a vendor product for a Network Element [i.3], an xApp or rApp which is sold/delivered to an operator.

Application Artifact: A file contained within an archive such as the Application Package.

Application Package: The set of information provided by a Solution Provider to a Service Provider representing all or a component of an Application.

Onboarding: The process of validating and ingesting an Application Package to make it available to other services within the SMO.

Service Provider: The RAN service provider often referred to as the operator.

Solution Provider: Application development entity often referred to as a vendor.

xApp: An application designed to run on the near-RT RIC as described in the O-RAN Architecture Description [i.2].

3.2 Symbols

For the purposes of the present document, the symbols given in 3GPP TR 21.905 [i.1] apply.

NOTE: A symbol defined in the present document takes precedence over the definition of the same symbol, if any, in 3GPP TR 21.905 [i.1].

3.3 Abbreviations

For the purposes of the present document, the abbreviations given in 3GPP TR 21.905 [i.1] and the following apply.

NOTE: An abbreviation defined in the present document takes precedence over the definition of the same abbreviation, if any, in 3GPP TR 21.905 [i.1].

DME	Data Management and Exposure
FCAPS	Fault, Configuration, Accounting, Performance, and Security
OAM	Operations, Administration, and Maintenance
RLBS	Replicated Load-Balanced Services
SME	Service Management and Exposure

4. Application Lifecycle Management (LCM)

4.1 Onboarding in the software lifecycle

The application software lifecycle is described in the OAM Architecture [i.5], clause 6. The onboarding phase identifies the "Onboarding" activity. This specification will detail the process that an Application Package from a Solution Provider is delivered to the Service Provider through the "Marketplace" external Meet Me Point.

Applications should be onboarded in a common manner, regardless of how they are deployed. The Application Package contents are dependent on the type of application being onboarded. The required artifacts and their format are defined in a separate specification, currently under development. The conventions used to convey the details of Onboarding activity are described in the OAM Architecture [i.5], figure 6.1-4.

4.2 Onboarding activity

The fine-grained activities of the Onboarding are shown below in figure 4.2-1. Activities of the Onboarding Service which impact the content of the Application Package are highlighted in bold. All the functions of this activity are specific to the Onboarding Service.

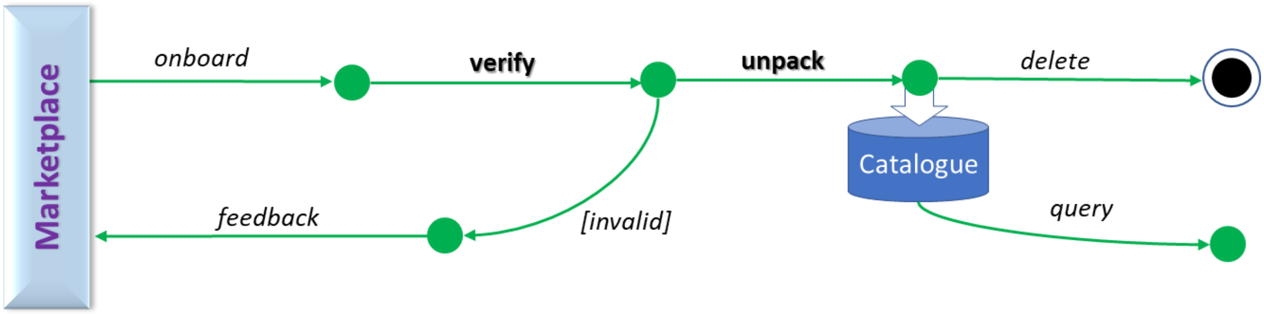

Figure 4.2-1: Activities of the Application Onboarding process

An Application Package is onboarded from the exchange, and its content verified. If valid, its contents (the App) are unpacked and stored/referenced in a catalogue. If invalid, the Service Provider may provide AppPackage-level feedback to the Solution Provider via the Marketplace. The capabilities described in this clause are the focus of the Onboarding SMOS.

5 General Aspects of the Onboarding SMOS

5.1 ETSI alignment

5.1.1 ETSI GS NFV SOL004 [2] alignment

Per the O-RAN Security Requirements [1] the package will follow the security requirements of ETSI GS NFV SOL004 [2]. However, SOL004 goes beyond just the security requirements and identifies a strict package artifact set. This set is not comprehensive enough to accommodate some Application Package capabilities required by O-RAN. Therefore, for ETSI based packages, ETSI GS NFV SOL004 [2] will be followed as specified. However, it is expected that there will be some augmentation of ETSI GS NFV SOL004 [2] or alternative approaches will be specified to cover O-RAN specific scenarios.

5.2 Onboarding as a distributed SMOS

5.2.1 Design patterns for Onboarding

The SMO may deploy one or more Onboarding SMOS instances based on the scale of Onboarding requests. The service assumes all instances are capable of onboarding a package. The FileSystem-As-A-Service can reduce physical data replication across service instance locations. However, it is expected that the Database-As-A-Service provides one consistent view of all Applications onboarded. Race conditions between services can exist if the same onboarding request is received independently to two different Onboarding SMOS instances. In this case resolution of the race condition is defined by the service implementation.

There are common design patterns for implementing a distributed service. A design pattern is a tried and tested implementation example which allows developers to follow a behavioral model that other developers have successfully implemented. Conditions such as whether the service consumer is centralized or decentralized are considered to determine which patter to use.

To provide greater flexibility and interoperability multiple patterns may be integrated. For the Onboarding SMOS where a user wants to onboard an Application Package, it is used from a centralized perspective, two common patterns exist one synchronous and one asynchronous that are adaptable to the Onboarding SMOS, the Replicated Load-Balanced Services (RLBS) and Saga patterns respectively.

5.2.2 Replicated Load-Balanced Services

RLBS is one of the simplest and most commonly used design patterns. All the service instances use a common/central load balancer. The first service instance would establish the load balancer and point to the virtual endpoint provided by the load balancer in the service registry. Each additional service instance would add itself to the back end of the load balancer. Simple service healthchecks can take service instances in and out of service providing robustness of the service when individual instances fail.

5.2.3 Saga

Saga provides an asynchronous pattern where instead of directy calling the API, a message is sent over an Event Bus. This eliminates the central controller and its single point of failure. It also self regulates any irregularities that may be induced due to the load balancing algorithm. However, it may become hard to debug when errors occur due the non-deterministic assignment of work to an instance.

5.3 Messaging

5.3.1 Procedure requests

In this messaging model the subscriber(s) is one the Onboarding SMOS. There could be potentially many publishers, Onboarding Service Consumers. The Onboarding SMOS will create the data product within the DME for the data structure to create an oboarding service request. Since some procedures are privileged and if supported through a notification then a form of authentication will have to be employed to ensure the requestor has the authorization to execute the procedure,

5.3.2 Procedure responses

In this messaging model the publisher(s) is the Onboarding SMOS. There could be potentially many subscribers, Onboarding Service Consumers. The Onboarding SMOS will create the data product within the DME for the responses to onboarding service requests.

5.3.3 Published notifications

In this messaging model there is one publisher, the Onboarding SMOS. There could be potentially many subcribers, Onboarding Service Consumers. The Onboarding SMOS will create the data product within the DME for the event(s) which will notify service consumers of state changes to Application Packages,

5.4 Reference vs copy

The Onboarding Service provides reference to objects which may require modification prior to deployment and some which should never be modified. Some artifacts may be small and others extremely large. The strategy for data reference vs data replication should be documented.

Large extremely static objects should only be replicated as needed for resiliency. Software binaries whether as a software update package or as an executable image in a cloud environment shouldn't be replicated for any other reason. Preferably the O-Cloud would have access to such a repository and therefore only the image reference would be registered with the O-Cloud for which it will receive a deployment request.

Other data may or may not require modification prior to being able to deploy the application. When data has to be modified it is done in the local copy maintained by the Onboarding Consumer. When no modification is required and the Onboarding Consumer needs to provide the artifact to another entity it is preferable that it provides a reference to the original artifact rather than a copy to it. This limits the number of copies that must be managed when an application version is no longer required and SMO storage space needs to be reclaimed.

5.5 General Aspect: Idempotence

The overall concept of idempotence is that repeated requests for the same thing will result in the same outcome. This has some limitations when applied to the public procedures of the Onboarding service which are discussed here.

The Onboarding procedure can be called multiple times to recover from procedural failures. But once the notification that the package has been made available to other SMOS, who may now reference the package data, reloading the package is not possible. To reload after this point, delete, due to scheduled delete or forced delete, is required before the package can be re-onboarded.

Deprecate can be called repeatedly. If the Application Package is already in a DEPRECATED state no change to the record or its scheduled deletion time will occur, but success will still be returned to the consumer.

Forced Delete causes the entry to no longer exist. Therefore, if the object does not exist it assumes it was previously deleted and instead of indicating a not found error, it will return an indication of success.

Repeated calls to Cancel Delete will not change the state which would have been moved to AVAILABLE or modify the already cleared scheduled deletion time attribute. However, success will still be returned.

Query is by default idempotent since is provides only read access to the data and no change occurs.

5.6 Role based access

The Onboarding service supports two roles, an operator user and an administrator user. To ensure that only entitled users are allowed to execute the procedures exposed by the Onboarding SMOS, the required role for each procedure is to be specified.

5.7 Deprecation vs deletion

The Onboarding SMOS provides data which may be referenced by other SMOS. It is not aware of all the references that might exists. Therefore, it cannot ensure that all references have been deleted, or can be deleted without violating the referential integrity of the data in the SMO. Therefore, it uses the concept of deprecation to identify Application Packages which have been targeted for delete. Onboarding event consumers would be notified when an Application Package has moved to the DEPRECATED state. The event will also reflect the time at which the entry will actually be deleted. Consumers with references to the data will need to remediate its reference to the data prior to the identified deletion date.

The package details for a DEPRECATED package can be retrieved. Consumers may retrieve artifacts that may be needed for recovery of applications already deployed. Packages in a DEPRECATED state should not be retrieved except as specified above. Additionally, the administrator will need to retrieve the package detail (metadata) in order verify if it is eligible for a forced delete.

A forced delete operation assumes that all the references to the data targeted for deletion in any SMOS has been previously deleted ensuring the overall referential integrity. For Applications Packages with a scheduled deletion any time before the deletion data a privileged user can cancel the deletion and thus move the Application Package back to the AVAILABLE state.

5.8 Hierarchical packages

Solution providers need to deliver a variety of application packages to service providers corresponding to different types of applications, e.g. Cloudified NFs, PNFs, rApps, etc.

In a cloud native paradigm, applications are usually made up of smaller software pieces, or microservices, with independent life cycles. For a Cloudified NF [i.4], this model is supported by the NF Deployment concept [i.4]. An NF Deployment can for example represent one microservice or a set of microservices that have tight interdependencies and are therefore bundled together by the solution provider.

When the solution provider needs to deliver a modification in one of these NF Deployments it is not necessary to deliver the complete Cloudified NF again but only the package of the affected NF Deployment.

Notwithstanding this, there is still information that belongs to the Cloudified NF as a whole, not to any of its constituent NF Deployments. An obvious example is a descriptor or artifact indicating how the cloudified NF is made up of NF Deployments. This type of information is delivered in a Cloudified NF package.

Thus, the solution provider uses packages at different levels to deliver the application software and artifacts: a Cloudified NF package and one or multiple NF Deployment packages, where the former references the latter ones. The reference is typically version specific.

It is expected that the Onboarding SMOS will be capable of performing the onboarding of a Cloudified NF package and the onboarding of NF Deployments packages as individual procedures.

6 Principles of the Onboarding SMOS

6.1 Contained functionality of the Onboarding SMOS

6.1.1 Package security

6.1.1.1 Application Package supply chain security

The Onboarding SMOS provides the functionality to meet all the security requirements assigned by O-RAN Security Requirements [1], clause 5.3.2.1.1 to the Application Package. The Onboarding SMOS verifies that the package and/or contained artifacts were properly signed and the Solution Providers certificate is valid.

6.1.1.2 Application Package content security

The Onboarding SMOS provides the functionality to meet all the security requirements assigned by the O-RAN Security Requirements [1], clause 5.3.2.1.2 The Onboarding SMOS verifies that the package contents have not been modified since the package was created.

6.1.2 Package delivery format

6.1.2.1 Application Package archive compression

The Application Package is expected to be transported in a compressed format. The Onboarding SMOS provides the functionality to know how the received archive is uncompressed so that downstream consumers of the package artifacts are able to directly access an uncompressed version of the artifact.

6.1.2.2 Application Package archive structure

The Application Package will have a defined structure that allows the Onboarding SMOS to locate the different artifacts. When the Onboarding SMOS extracts the artifacts and stores them in catalogues the package structure is not maintained. Once unpacked, individual artifacts are identifiable without the support of the package structure. Thus, consumers of individual artifacts are in general unaware of the original package structure and are not affected by changes in it.

6.2 Artifact handling

6.2.1 File data

6.2.1.1 File storage

File Data refers to the file content, which from the perspective of the Onboarding SMOS is opaque. It does not mean that the content of the data is not organized, only that it is not from the perspective of the Onboarding SMOS. A configuration file in YANG has a specific structure and definition, as does a configuration file in JSON. However, from the Onboarding SMOS perspective, just like all the artifacts in an Application Package they are just a file and needs to be stored.

6.2.1.2 File security

Images and Software Packages are specific artifacts with specific security requirements in O-RAN.

6.2.1.3 File accessibility

Some files can be extremely large and therefore might be in a common tool accessible to the SMO and O-Clouds alike. The Onboarding service will provide an accessible URL or URN to the file. Therefore, the consumer need not know the details of where or how it is stored.

6.2.2 Catalog data

There is metadata associated with the package and its artifacts. This metadata is stored in a query-able catalog. Consumers do not need to know what database or storage technology is used for the catalog as the Onboarding Service will provide a Query procedure to filter and select the data they need for their processing.

7 Onboarding functionality

7.1 Use cases

7.1.1 Use case overview

The use cases below are high level descriptions from the perspective of the Onboarding Service Consumer. The use case identifies several aspects of the service behaviour the following variable are used to express the use case.

Variable Text	Description
Consumer	Who is requesting an operation to be performed by the Onboarding Service
Feature Name	What feature or capability is being requested
Feature Behavior	What extending behaviors does the feature need to provide, can be left blank
Expected Outcome	What is the expected outcome of the feature
Enabled Need	Why does the consumer need the feature

The variables can be inserted into an epic like story:

As the **{Consumer}** I need the Onboarding Service to provide the capability to **{Feature Name}** [with the ability to **{Feature Behavior}**] to **{Expected Outcome}** so that I can **{Enabled Need}**.

7.1.2 Onboard

Variable	Value
Consumer	Service Designer
Feature Name	Onboard an Application
Feature Behavior	Recover from previous failures
Expected Outcome	Provide package artifacts in a catalogue
Enabled Need	Certify and/or train the Application as part of a service.

7.1.3 Deprecate

Variable	Value
Consumer	Service Designer
Feature Name	Deprecate an Application
Feature Behavior	Identify its scheduled deletion date
Expected Outcome	Change the Application Package state is DEPRECATED and establish a scheduled deletion date
Enabled Need	Retire older application versions once they are no longer needed by operations

7.1.4 Forced Delete

Variable	Value
Consumer	Systems Administrator
Feature Name	Force the deletion of an Application
Feature Behavior	Without regard to referential integrity
Expected Outcome	Application and its artifacts are forcibly deleted from the catalogue
Enabled Need	Cleanup Applications in the catalogue

7.1.5 Application List Query

Variable	Value
Consumer	Service Designer
Feature Name	Query for a list of available Applications
Feature Behavior	To filter by Application type, name, vendor, and/or version
Expected Outcome	A list of qualifying Applications can be identified
Enabled Need	The selection of an Application to retrieve

7.1.6 Application Manifest Query

Variable	Value
Consumer	Service Designer
Feature Name	Query Application Manifest
Feature Behavior	To filter to exclude or include a list of artifacts
Expected Outcome	Application descriptive data and/or a list of qualifying artifacts each identified as a link to a readable file
Enabled Need	To get package data not included in an artifact

7.1.7 Application Artifact Query

Variable	Value
Consumer	Service Designer
Feature Name	Query Application Artifacts
Feature Behavior	To filter a list of artifacts by by type or use
Expected Outcome	Application list of qualifying artifacts identified as a link to a readable file

Enabled Need	Open an artifact as a file for processing

7.1.8 Application Delete Cancelation

Variable	Value
Consumer	Systems Administrator
Feature Name	Cancel Application Delete
Feature Behavior	
Expected Outcome	Change the Application Package state to AVAILABLE and remove a scheduled deletion date
Enabled Need	Cancel a scheduled delete to extend the availability of the Application Package

7.2 Procedures

7.2.1 Procedure actors

The following actors are used in the analysis as consumers of information. Other actors may be defined in 6.2.1 of the OAM Architecture [i.5].

Table 7.2.1-1 Actor symbols, roles, and descriptions for procedure interaction diagrams

Actor	Role	Description
	Application Package Source	The Application Package Source is the Application Archive outside the SMO but accessible from the SMO. Although the package is a file on the source and therefore is like an entity a "Participant" UML symbol is used since it is external to the Onboarding SMOS.
	Service Consumer	An Onboarding SMOS Consumer who initiates the "Onboarding" process to an accessible Application Package. The Onboarding Service Consumer can also represent a consumer of an Onboarded Application Package or one or more of its artifacts. The "Participant" UML symbol represents any type of participant participant that is external to the Onboarding SMOS.
	An Event Bus	The Event Bus is used to publish and subcribe to events. The "Participant" UML symbol is used to represent the service is outside the Onboarding SMOS.
	The Onboarding SMOS API	This is the Onboarding SMOS API. The "Boundary" UML symbol is reserved for Onboarding SMOS components and represents the point from which a consumer can invoke a service capability. Other than user privilege check there is no service logic within the API and it must call a service procedure to implement the exposed capability.

	An Onboarding SMOS Service Procedure	This can be any Onboarding SMOS procedure which implements service logic. The "Control" UML symbol is reserved for Onboarding SMOS components and represents a capability provided by the service which may or may not be exposed through the Onboarding API. As a Control it can call any other participant.
	File System Store for Application Package and its Artifacts	This is a File System for the Onboarding SMOS storage of files. While the Onboarding SMOS is the only writer of files to this entity, external consumers can read files placed there. The "Entity" UML symbol is used here to represent a collection of files which are internally managed as a component of the Onboarding SMOS. As an Entity it cannot call any other participant. It can only be called.
	Onboarding Catalog	This is a queryable data store which allows selection based on field value filtering and relatioinships between data elements. The "Entity" UML symbol is used to identify it as a persistent structured store as a component of the Onboarding SMOS. As an Entity it cannot call any other participant. It can only be called.

7.2.2 Exception handling

When any of the following exceptions occur, normal processing terminates.

Exception	Description	Occurring Public Procedure
Pre-Existence Failure	Attempt to onboard an existing package that has already been made available to other SMOSs. Before re-onboarding it must be deleted first. This will require assistance from a user with the administrative role.	Onboarding
Authentication Failure	Application Package is not properly "signed" by the Solution Provider. The vendor's certificate was not used to "sign" the package therefore the source of the package is suspect, and a new, properly signed package should be received from the vendor before retrying to onboard.	Onboarding
Verification Failure	Application Package artifacts appear to have been modified and do not match the security data inside the package. The package should be considered damaged or tampered. The package should be considered suspect, and a new package should be received from the vendor before retrying to onboard.	Onboarding
File Storage Error	Unknown cause of failure to the SMO file system. This could be due to storage, permissions, or some other file system failure. Onboarding can be retried. If the problem persists, consult with an administrator.	Onboarding
Database Add Failed	The Application Package record could not be added to the structured data store. Onboarding can be retried. If the problem persists, consult with an administrator.	Onboarding

Package Not Found	requested Package did not exist or was in a state in which it could not be returned.	Query
		Deprecated Delete
		Forced Delete
		Cancel Deprecation
Permission Error	The user did not have a required role.	Forced Delete
		Cancel Deprecation
		Modify Retention Period
File Delete Error	Unknown cause of failure to the SMO internal file system during the delete process. The file was not deleted.	Forced Delete
Update Configuration Error	The service configuration could not be updated, created or deleted due to an unknown error.	Configuration Update

7.2.3 Procedure: API Handling

7.2.3.1 Description

The invocation of a service exposed by the Onboarding SMOS can be realized via a synchronous method or asynchronous method. The difference between both methods is illustrated in figure 7.2.3.2-1. The methods are common for all service invocations and therefore they are only documented in this clause. The procedure flows simply show the invocation of the service consumer as an interaction with the API Boundary, i.e. the synchronous method. This does not restrict the ability of the service consumer to choose the asynchronous method for the service invocation.

7.2.3.2 Procedure

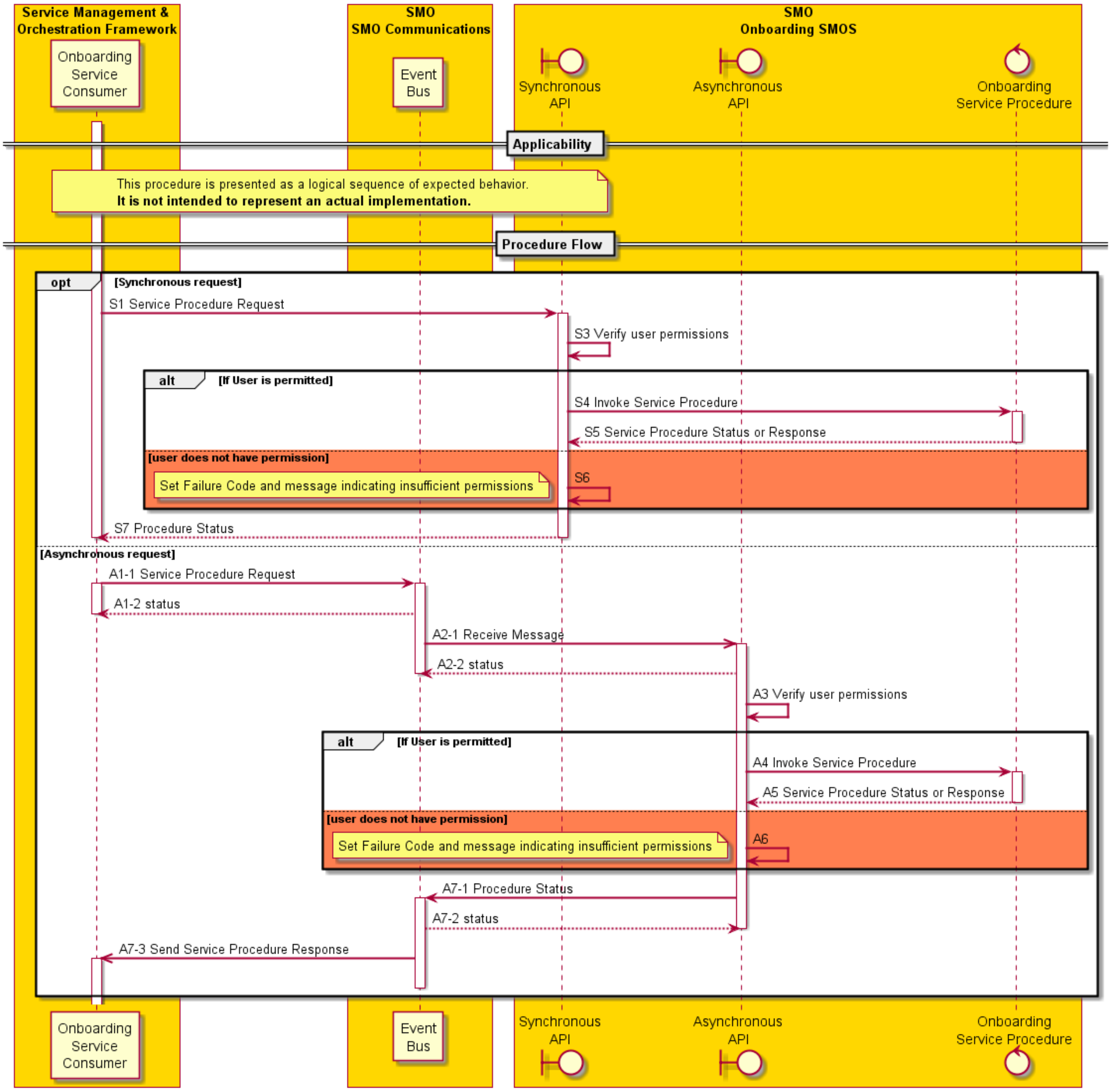

Figure 7.2.3.2-1 API Handling interaction diagram

7.2.4 Procedure: Onboard

7.2.4.1 Onboard

7.2.4.1.1 Description

The Onboard procedure is a public procedure that can be invoked by a general or administrative user via the onboarding API. The procedure checks if the package to be onboarded has been tried before and if its present state allows, it restores the environment to a condition conducive to proper operation of the end-to-end flow of the Onboard, Verify, and Unpack procedures.

NOTE: There can be a multitude of fault conditions during the procedure beyond those illustrated in the example, In particular the flow does not show the detection of concurrent requests to onboard the same package which may be required to ensure that concurrent requests do not corrupt the catalog or file store.

7.2.4.1.2 Procedure

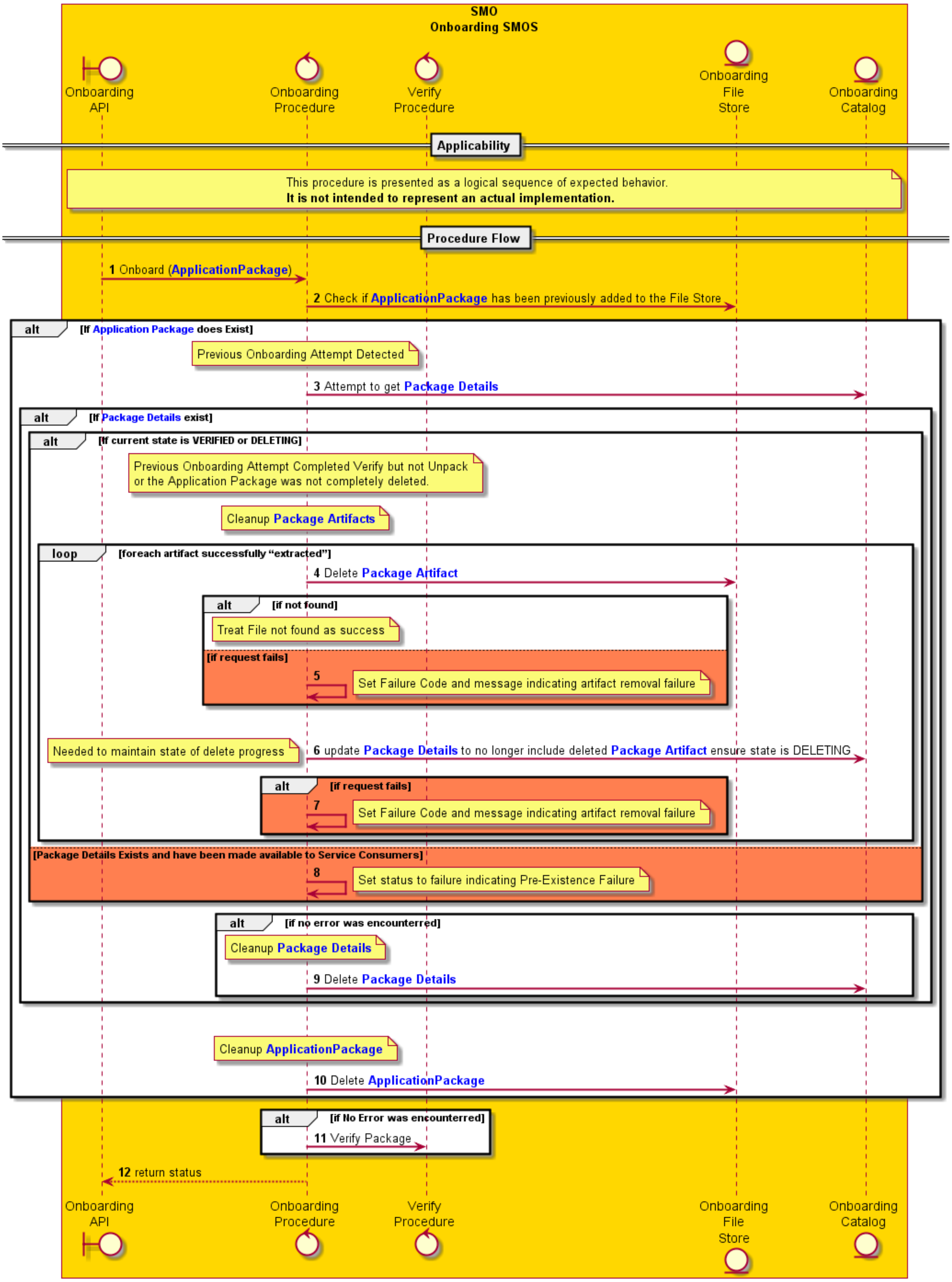
SMO
Onboarding SMOS
Onboarding API
Onboarding Procedure
Verify Procedure
Onboarding File Store
Onboarding Catalog
Applicability
This procedure is presented as a logical sequence of expected behavior.
It is not intended to represent an actual implementation.
Procedure Flow
1 Onboard (ApplicationPackage)
2 Check if ApplicationPackage has been previously added to the File Store
alt [If Application Package does Exist]
Previous Onboarding Attempt Detected
3 Attempt to get Package Details
alt [If Package Details exist]
alt [If current state is VERIFIED or DELETING]
Previous Onboarding Attempt Completed Verify but not Unpack
or the Application Package was not completely deleted.
Cleanup Package Artifacts
loop [foreach artifact successfully "extracted"]
4 Delete Package Artifact
alt [if not found]
Treat File not found as success
[if request fails]
5
Set Failure Code and message indicating artifact removal failure
Needed to maintain state of delete progress
6 update Package Details to no longer include deleted Package Artifact ensure state is DELETING
alt [if request fails]
7
Set Failure Code and message indicating artifact removal failure
[Package Details Exists and have been made available to Service Consumers]
8
Set status to failure indicating Pre-Existence Failure
alt [if no error was encounterred]
Cleanup Package Details
9 Delete Package Details
Cleanup ApplicationPackage
10 Delete ApplicationPackage
alt [if No Error was encounterred]
11 Verify Package
12 return status
Onboarding API
Onboarding Procedure
Verify Procedure
Onboarding File Store
Onboarding Catalog

Figure 7.2.4.1.2-1 Onboard procedure interaction diagram

7.2.4.2 Procedure: Verify

7.2.4.2.1 Description

The Verify procedure is a private procedure which encapsulates the security aspects of the onboarding process and can only be invoked by the Onboarding procedure. As a private procedure its implementation as a procedure is at the discretion of the Solution Provider. It is described here to establish the Information Model and functional requirements of the end-to-end Onboarding procedure.

7.2.4.2.2 Procedure

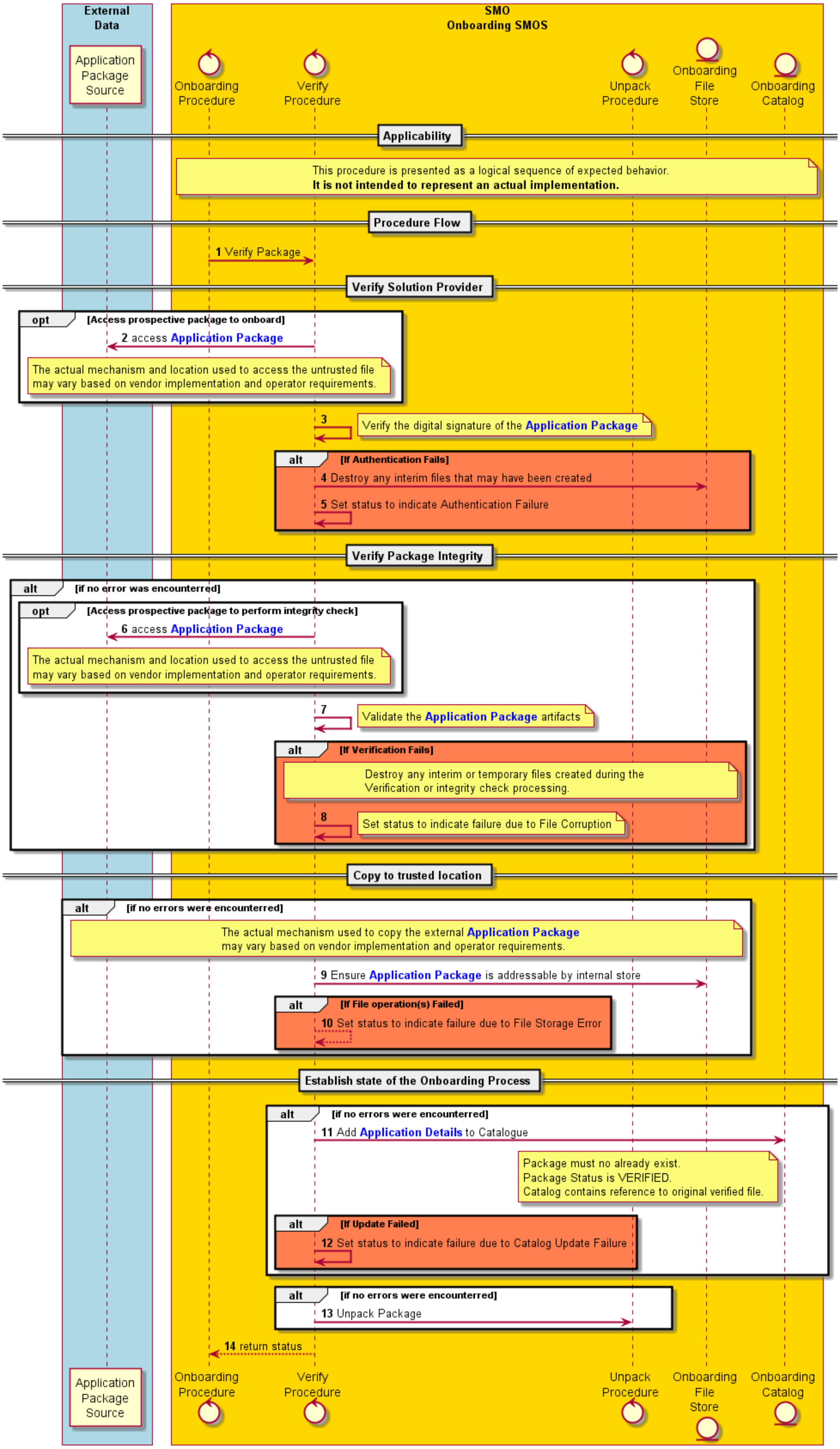
External Data
Application Package Source
SMO
Onboarding SMOS
Onboarding Procedure
Verify Procedure
Unpack Procedure
Onboarding File Store
Onboarding Catalog
Applicability
This procedure is presented as a logical sequence of expected behavior.
It is not intended to represent an actual implementation.
Procedure Flow
1 Verify Package
Verify Solution Provider
opt [Access prospective package to onboard]
2 access Application Package
The actual mechanism and location used to access the untrusted file may vary based on vendor implementation and operator requirements.
3
Verify the digital signature of the Application Package
alt [If Authentication Fails]
4 Destroy any interim files that may have been created
5 Set status to indicate Authentication Failure
Verify Package Integrity
alt [if no error was encounterred]
opt [Access prospective package to perform integrity check]
6 access Application Package
The actual mechanism and location used to access the untrusted file may vary based on vendor implementation and operator requirements.
7
Validate the Application Package artifacts
alt [If Verification Fails]
Destroy any interim or temporary files created during the Verification or integrity check processing.
8
Set status to indicate failure due to File Corruption
Copy to trusted location
alt [if no errors were encounterred]
The actual mechanism used to copy the external Application Package may vary based on vendor implementation and operator requirements.
9 Ensure Application Package is addressable by internal store
alt [If File operation(s) Failed]
10 Set status to indicate failure due to File Storage Error
Establish state of the Onboarding Process
alt [if no errors were encounterred]
11 Add Application Details to Catalogue
Package must no already exist.
Package Status is VERIFIED.
Catalog contains reference to original verified file.
alt [If Update Failed]
12 Set status to indicate failure due to Catalog Update Failure
alt [if no errors were encounterred]
13 Unpack Package
14 return status

Figure 7.2.4.2.2-1 Verify procedure interaction diagram

7.2.4.3 Procedure: Unpack

7.2.4.3.1 Description

The Unpack procedure is a private procedure which encapsulates the aspects of compression and archive structure aspects of the onboarding process. The Unpack procedure can only be invoked by the Verify procedure if it completes without error. As a private procedure its implementation is at the discretion of the Solution Provider. It is described here to establish the requirements of the end-to-end Onboarding procedure.

7.2.4.3.2 Procedure

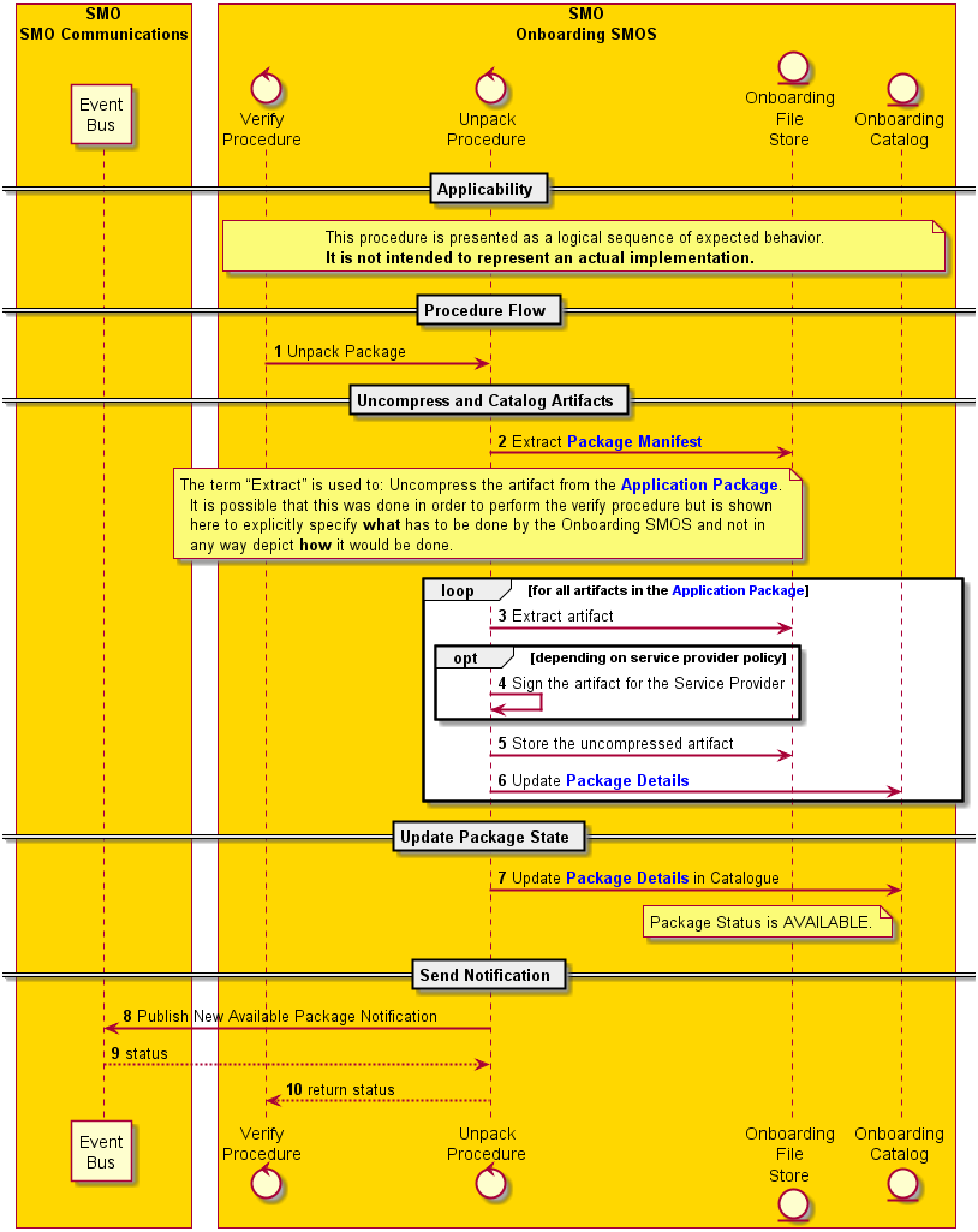

Figure 7.2.4.3.2-1 Unpack procedure interaction diagram

7.2.4.4 Procedure Onboard, verify, unpack: alternative example flow

7.2.4.4.1 External vs. internal behaviour

The flows in clauses 7.2.4.1, 7.2.4.2 and 7.2.4.3, when combined, illustrate a possible way of sequencing the different actions expected from the Onboarding SMOS Producer during the onboarding of a package, including the verification of the integrity and authenticity of the package, the unpacking and the storing of the contents in a File Store. They also include error handling, in particular, in the case of an attempt to onboard a package that was already onboarded and in state AVAILABLE.

However, as many of the actions are internal to the Onboarding SMOS Producer, the flows are not meant to represent mandatory sequences.

The flows fulfil the functional requirements of the Onboarding SMOS Producer during the onboarding procedure with a sequence of interactions. One can distinguish:

- interactions of the Onboarding SMOS Producer with the Onboarding SMOS Consumers and the end result of the onboarding procedure
- internal interactions within the Onboarding SMOS Producer

The interactions with the consumer and the end result of the procedure define the externally visible behaviour of the Onboarding SMOS Producer. An Onboarding SMOS Producer has to fulfil the functional requirements of the onboarding procedure. It also has to expose the externally visible behaviour and end result, as specified in the flows, in order to ensure interoperability.

The internal interactions are just a possible way, among others, for an Onboarding SMOS Producer to fulfil the functional requirements and expected external behaviour and they don't constitute a mandatory sequence.

The onboarding procedure starts with an interaction with a consumer:

- Onboard service request

The successful result of the onboarding of a package is:

- Integrity and authenticity of the package and artifacts have been verified
- All the artifacts in the onboarded package, included the manifest, stored in the Onboarding File Store
- Package and artifact records created in the Onboarding Catalog
- State of the package set AVAILABLE
- Notification sent to consumers that have subscribed to notifications

Furthermore, an onboarding request attempting to onboard a package that is already onboarded and in state AVAILABLE is not allowed and results in an error that is returned to the consumer.

All the interactions between the Onboarding SMOS Producer and the consumers and the result of the onboarding procedure as listed above constitute the standard behaviour of an O-RAN compliant Onboarding SMOS Producer during the onboarding procedure.

In contrast, all internal interactions shown in the flows can be subject to implementation alternatives that do not affect the end result.

In particular:

- Sequence of the internal verify and unpack procedures can be dependent on the security option used in the package. In some cases the verification requires unpacking of each single artifact. Therefore, an Onboarding SMOS Producer may opt for performing both internal procedures in parallel.

- The VERIFIED state is a transient state that in some valid sequences may not exist, for example if verifying and unpacking is done in parallel and the package state is directly set to AVAILABLE at the end of the procedure.
- The service logic distributed in the control procedures onboard, verify, unpack can be combined in a single control procedure or distributed differently
- There are alternative ways of handling concurrent attempts to onboard the same package. It is also a valid sequence to reject an attempt while there is another ongoing procedure related to the same package.
- The construction of the artifact records may be undertaken by the Onboarding Catalog after retrieving the manifest from the Onboarding File Store
- An Onboarding procedure may include other cleaning actions when encountering error situations apart from the ones shown in the flows.

The above list is not intended to be exhaustive.

To illustrate some of these differences an alternative and also valid sequence flow encompassing the complete onboard, verify and unpack is shown in figure 7.2.4.4.2-1.

NOTE: there can be a multitude of fault conditions during the procedure beyond those illustrated in the example, In particular the flow does not show the clean up actions that may be required upon detection of a previous unsuccessful onboarding attempt related to the same package.

7.2.4.4.2 Procedure

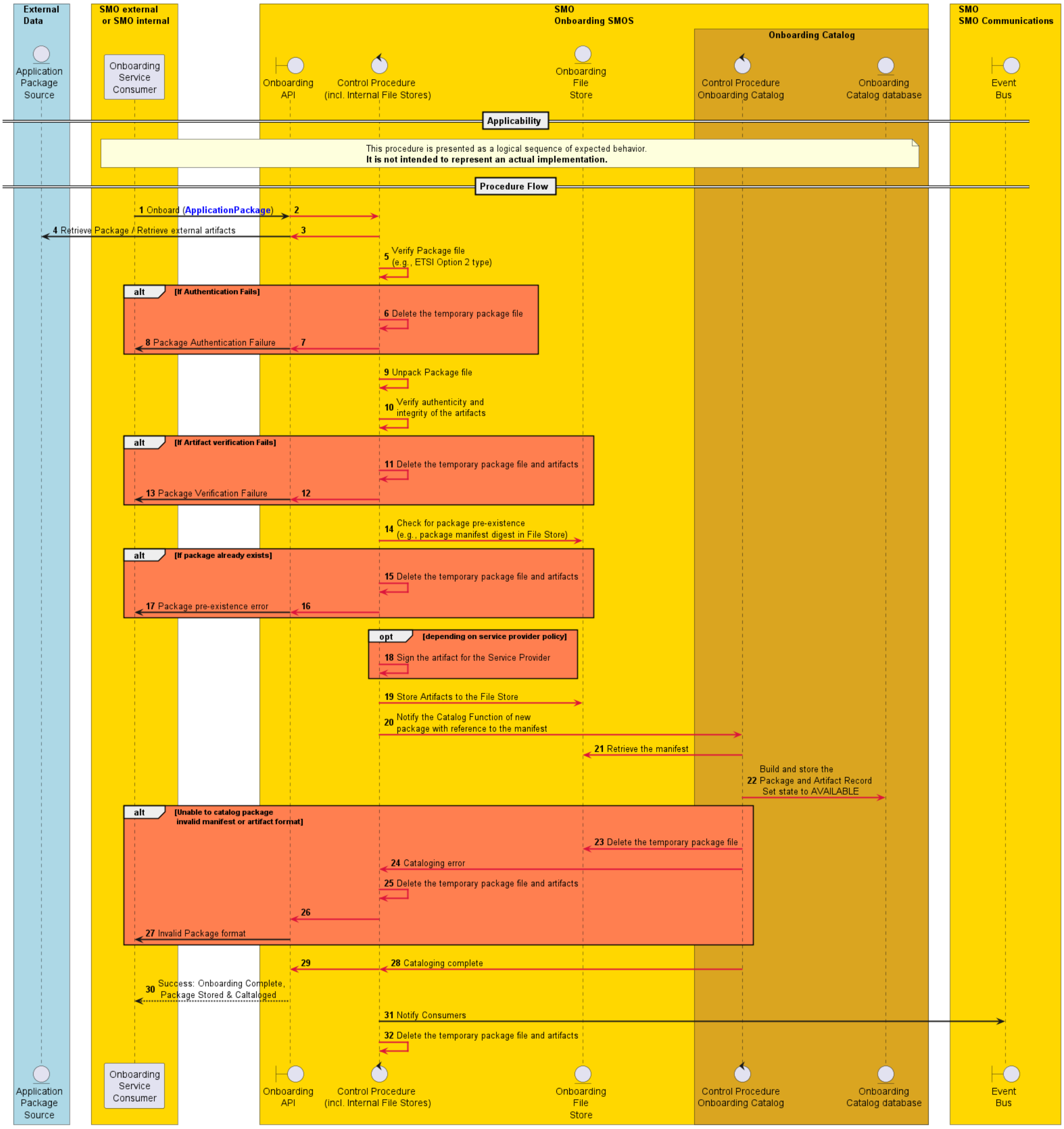

Figure 7.2.4.4.2-1 Onboard procedure alternative example interaction diagram

7.2.5 Procedure: Configuration Query

7.2.5.1 Description

The Configuration Query procedure is a public procedure that can be invoked any user. The procedure provides exposure of the configuration parameters through the onboarding API.

7.2.5.2 Procedure

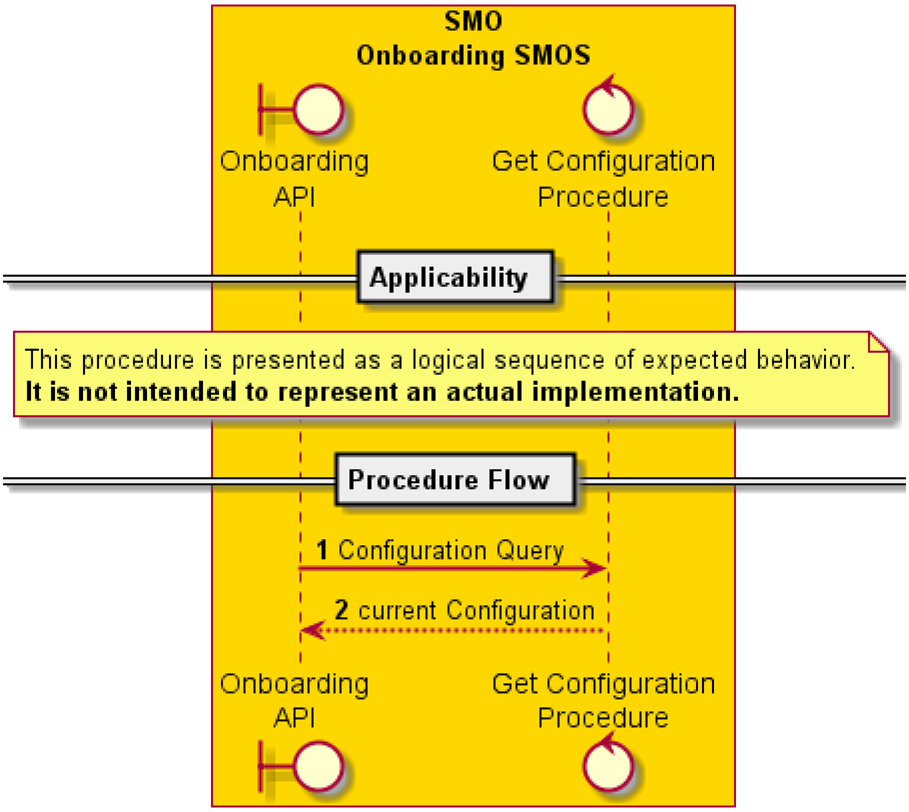

Figure 7.2.5.2-1 Configuration Query procedure interaction diagram

7.2.6 Procedure: Configuration Update Period

7.2.6.1 Description

The Configuration Update procedure is a public procedure that can only be invoked by an administrative user. The procedure provides exposure of the configuration parameters and allows the consumer to modify their values through the onboarding API. The Retention Period is the only configuration parameter identified by this specification, but others may exist based on the implementation.

7.2.6.2 Procedure

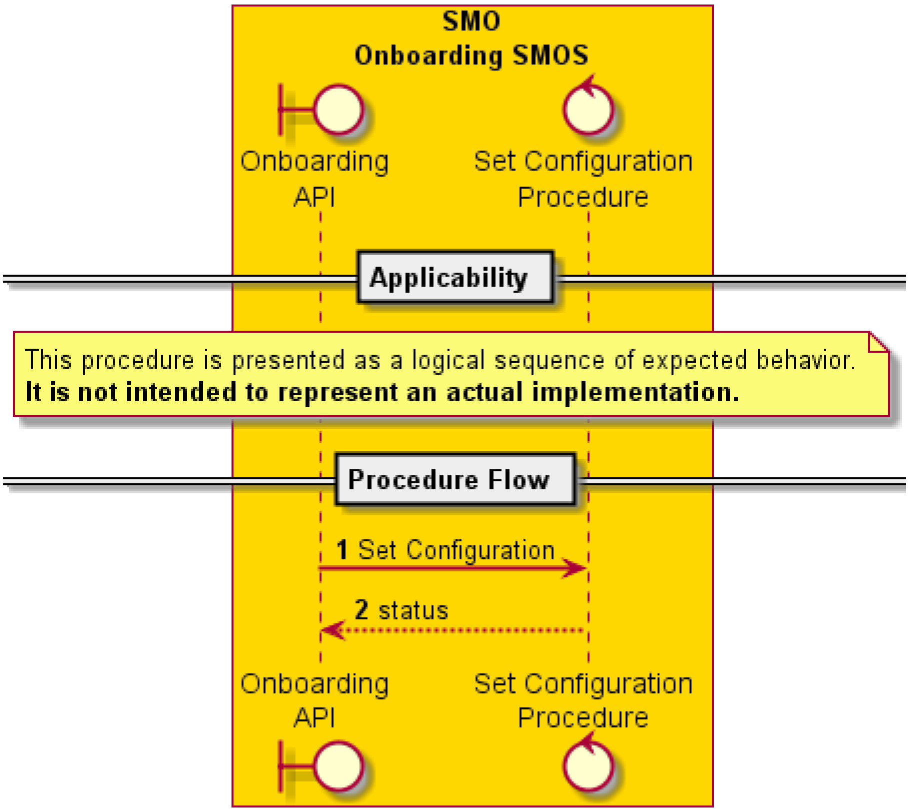

Figure 7.2.6.2-1 Configuration Update procedure interaction diagram

7.2.7 Procedure: Catalog Query

7.2.7.1 Description

The Query procedure is a public procedure that can be invoked by a general or administrative user through the onboarding API. It allows consumers to discover and access packages that have been successfully onboarded.

7.2.7.2 Procedure

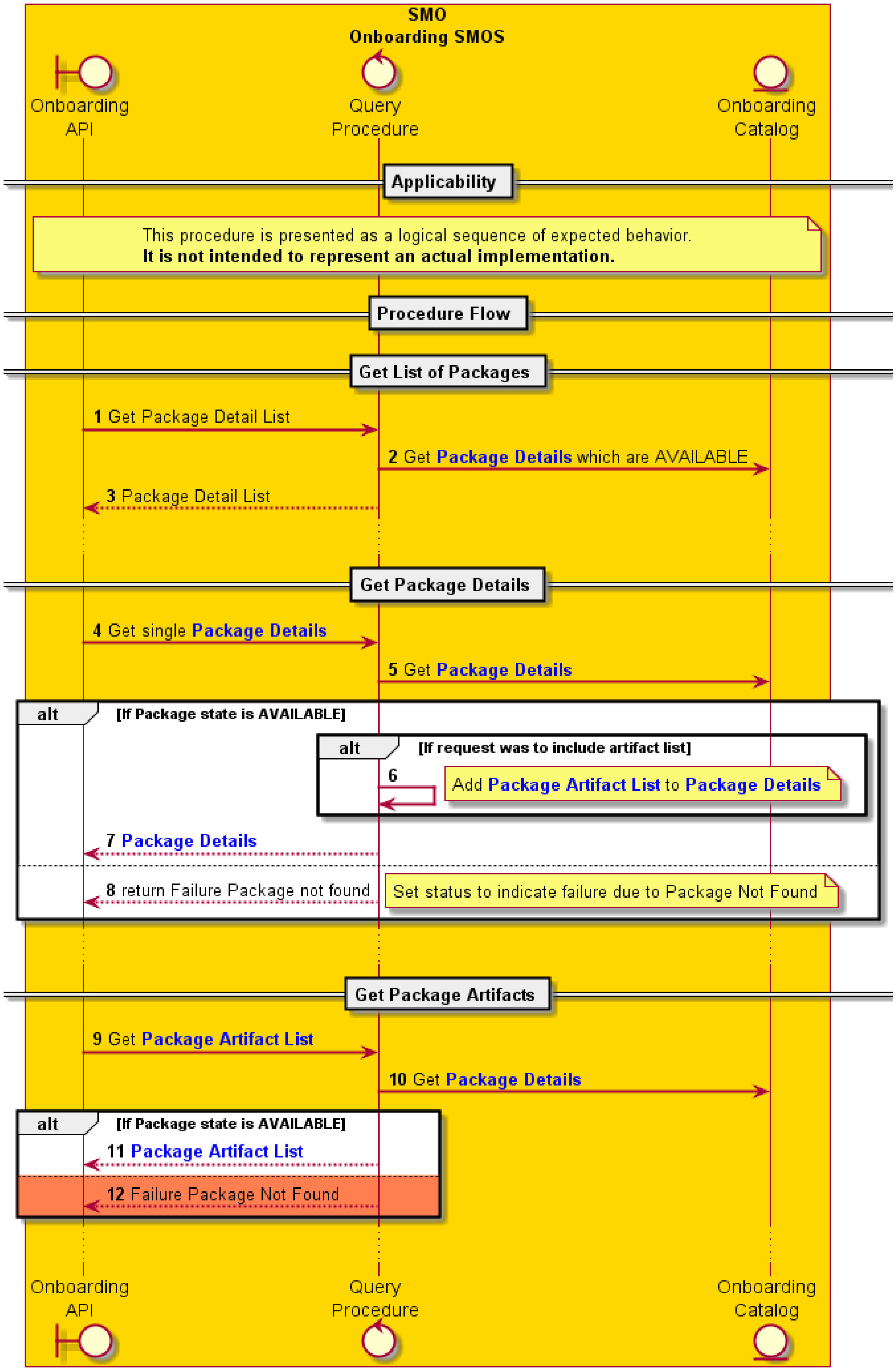
SMO
Onboarding SMOS
Onboarding API
Query Procedure
Onboarding Catalog
Applicability
This procedure is presented as a logical sequence of expected behavior.
It is not intended to represent an actual implementation.
Procedure Flow
Get List of Packages
1 Get Package Detail List
2 Get Package Details which are AVAILABLE
3 Package Detail List
Get Package Details
4 Get single Package Details
5 Get Package Details
alt [If Package state is AVAILABLE]
alt [If request was to include artifact list]
6
Add Package Artifact List to Package Details
7 Package Details
8 return Failure Package not found
Set status to indicate failure due to Package Not Found
Get Package Artifacts
9 Get Package Artifact List
10 Get Package Details
alt [If Package state is AVAILABLE]
11 Package Artifact List
12 Failure Package Not Found
Onboarding API
Query Procedure
Onboarding Catalog

Figure 7.2.7.2-1 Query procedure interaction diagram

7.2.8 Procedure: Artifact File Access

7.2.8.1 Description

The Artifact File Access procedure is a procedure that uses the file system APIs which are on the File Store that can be reached by the "filename" of the artifact which is a complete URL supporting the file I/O type of services.

7.2.8.2 Procedure

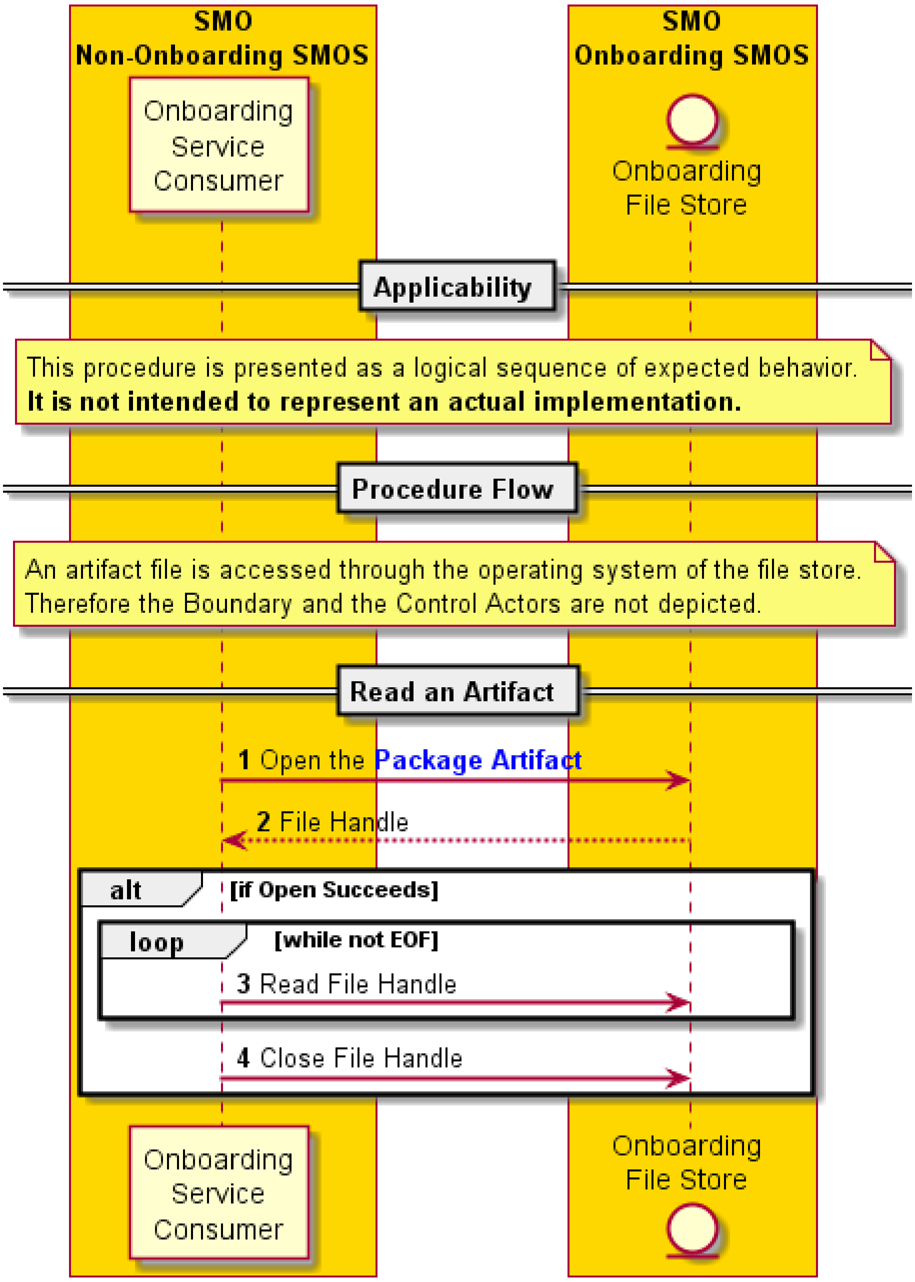

Figure 7.2.8.2-1 Artifact File Access procedures interaction diagram

7.2.9 Procedure: Deprecate Package

7.2.9.1 Description

The Deprecate Package procedure is a public procedure that can be invoked by an administrative user via the onboarding API. The procedure can only be performed at the Application Package level. If the package does not exist an error is returned. If the Application Package is already in a DEPRECATED state success is returned but no change is made. If the Application Package is AVAILABLE the package state is moved to DEPRECATED and the scheduled deletion time is established based on the current retention period configuration value and/or the date of deprecation is established.

7.2.9.2 Procedure

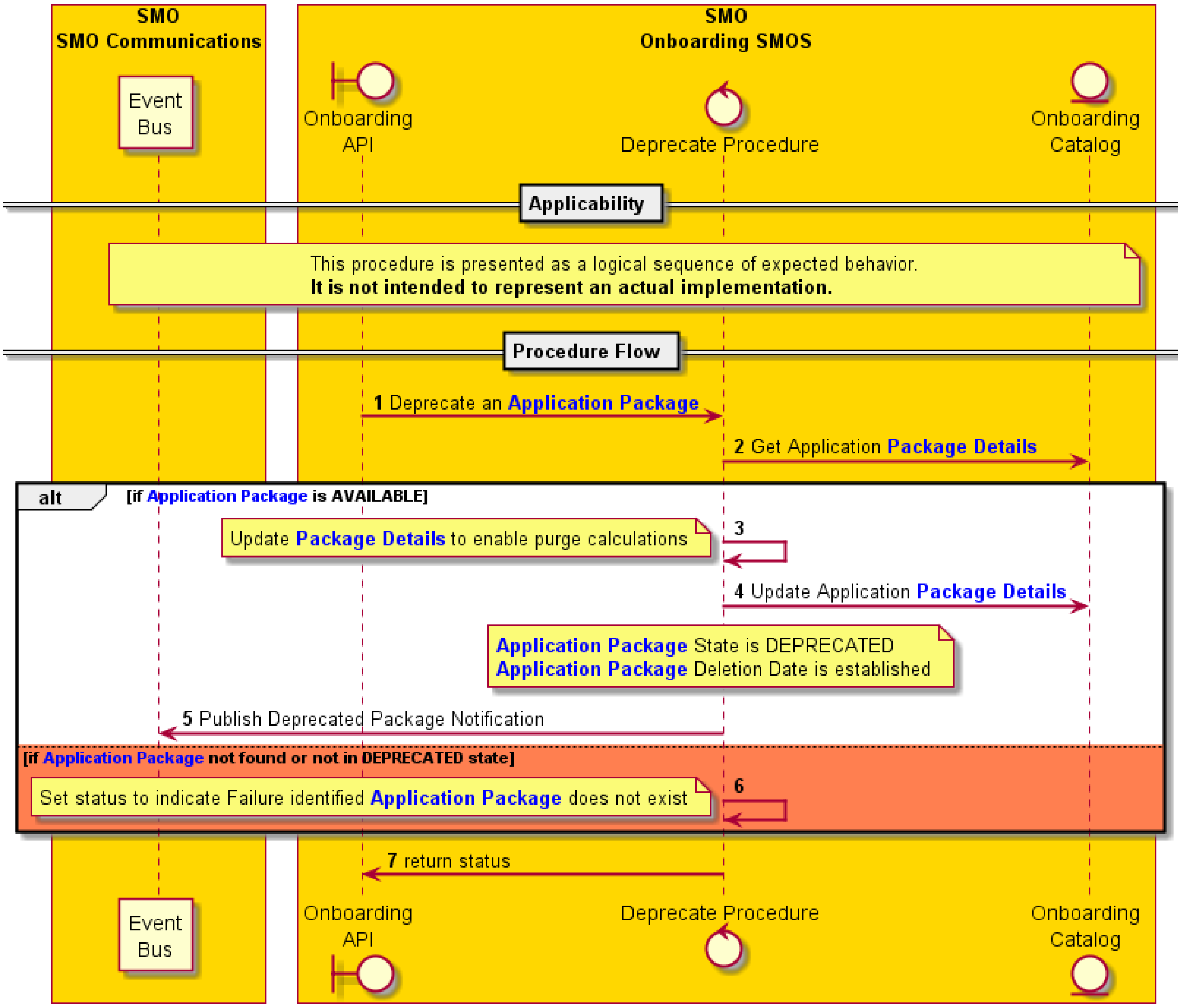

Figure 7.2.9.2-1 Deprecate Package procedure interaction diagram

7.2.10 Procedure: Cancel Deprecation

7.2.10.1 Description

The Cancel Deprecation procedure is a public procedure that can only be invoked by an administrative user through the onboarding API. The procedure can only be performed at the Application Package level. If an Application Package is inthe AVAILABLE state, then success is returned. If the Application Package is in a DEPRECATED state, then the Deletion Date is removed, and the Application State is changed to AVAILABLE before success is returned. If the Application Package is in any other state an error is returned.

7.2.10.2 Procedure

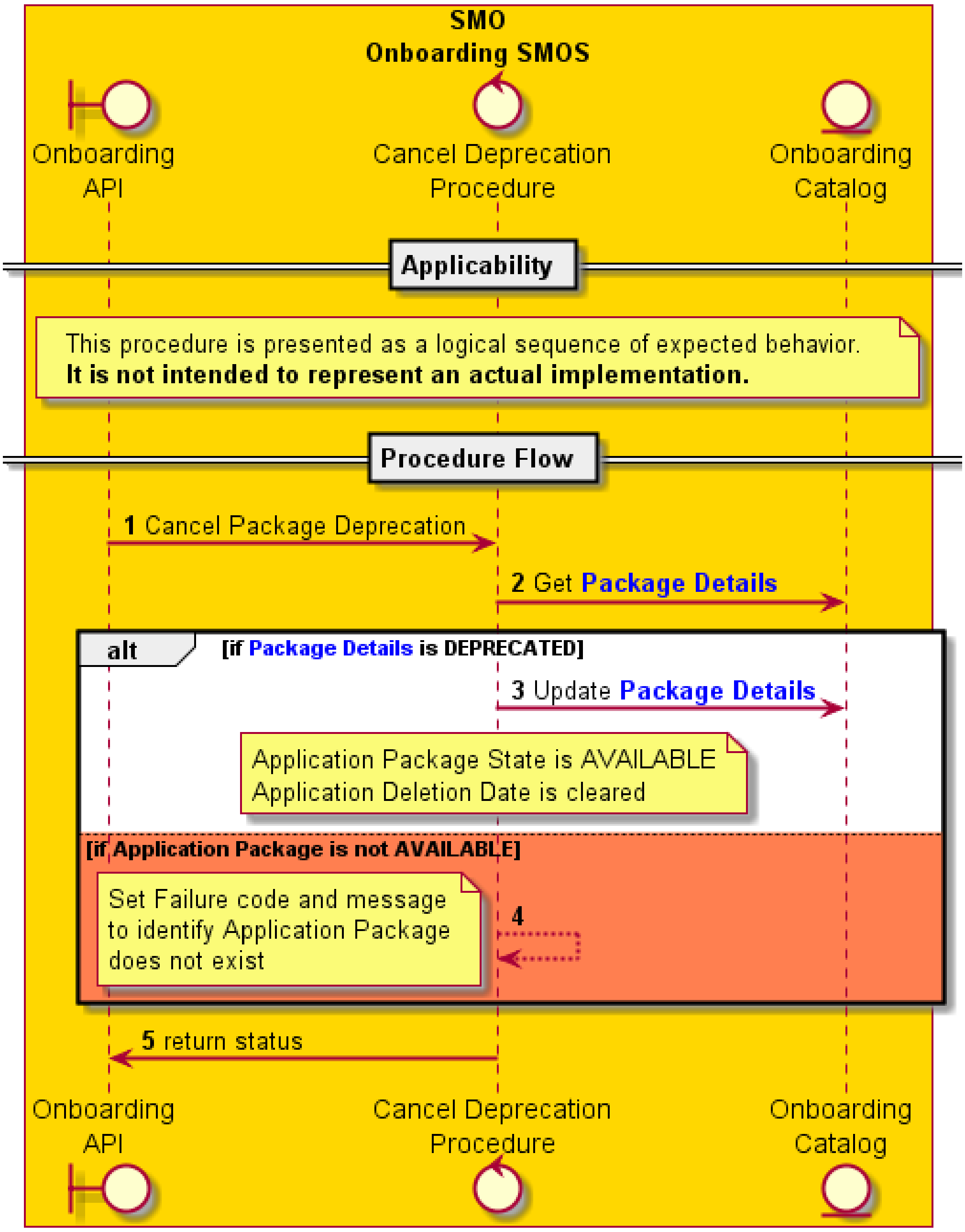

Figure 7.2.10.2-1 Cancel Deprecation procedure interaction diagram

7.2.11 Procedure: Forced Delete

7.2.11.1 Description

The Forced Delete procedure is a public procedure that can be invoked by an administrative user. The procedure is also invoked by the Purge procedure. The procedure can only be performed at the Application Package level. If the package does not exist success is still returned since the goal of the procedure was its removal. Upon successful completion the Application Package and all its artifacts are deleted from the Onboarding service.

7.2.11.2 Procedure

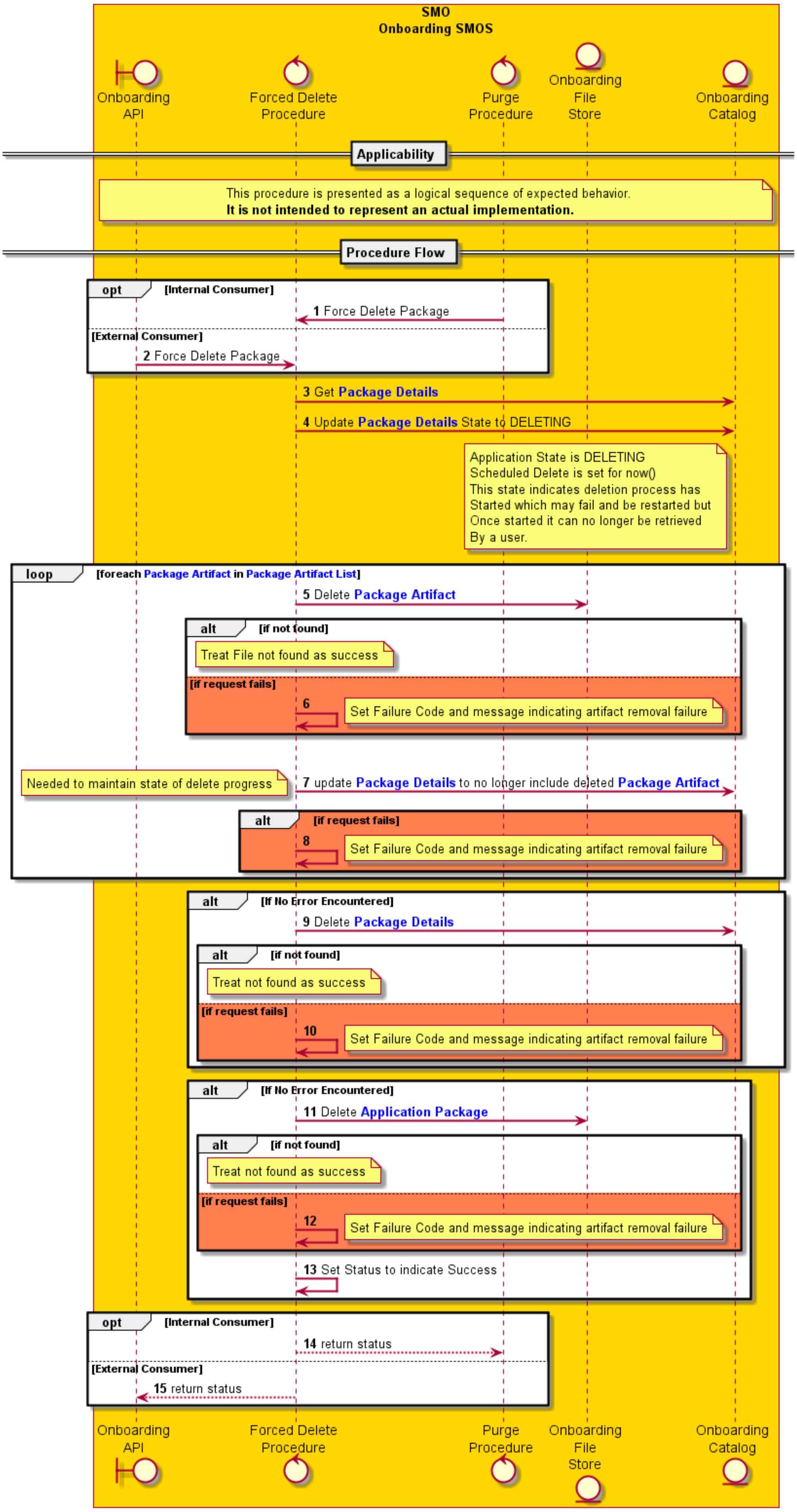
SMO
Onboarding SMOS
Onboarding API
Forced Delete Procedure
Purge Procedure
Onboarding File Store
Onboarding Catalog
Applicability
This procedure is presented as a logical sequence of expected behavior.
It is not intended to represent an actual implementation.
Procedure Flow
opt [Internal Consumer]
1 Force Delete Package
[External Consumer]
2 Force Delete Package
3 Get Package Details
4 Update Package Details State to DELETING
Application State is DELETING
Scheduled Delete is set for now()
This state indicates deletion process has
Started which may fail and be restarted but
Once started it can no longer be retrieved
By a user.
loop [foreach Package Artifact in Package Artifact List]
5 Delete Package Artifact
alt [if not found]
Treat File not found as success
[if request fails]
6
Set Failure Code and message indicating artifact removal failure
Needed to maintain state of delete progress
7 update Package Details to no longer include deleted Package Artifact
alt [if request fails]
8
Set Failure Code and message indicating artifact removal failure
alt [If No Error Encountered]
9 Delete Package Details
alt [if not found]
Treat not found as success
[if request fails]
10
Set Failure Code and message indicating artifact removal failure
alt [If No Error Encountered]
11 Delete Application Package
alt [if not found]
Treat not found as success
[if request fails]
12
Set Failure Code and message indicating artifact removal failure
13 Set Status to indicate Success
opt [Internal Consumer]
14 return status
[External Consumer]
15 return status
Onboarding API
Forced Delete Procedure
Purge Procedure
Onboarding File Store
Onboarding Catalog

Figure 7.2.10.2-1 Forced Delete procedure interaction diagram

7.2.12 Procedure: Purge

7.2.12.1 Description

The Purge procedure is a private procedure which encapsulates the auto-deletion of DEPRECATED or PARTIAL Application Packages. As a private procedure its implementation as a procedure is at the discretion of the Solution Provider and it is not invoked via the onboarding API. It is described here to establish the abstracted functionality and requirements hidden within the service implementation of the Onboarding SMOS.

7.2.12.2 Procedure

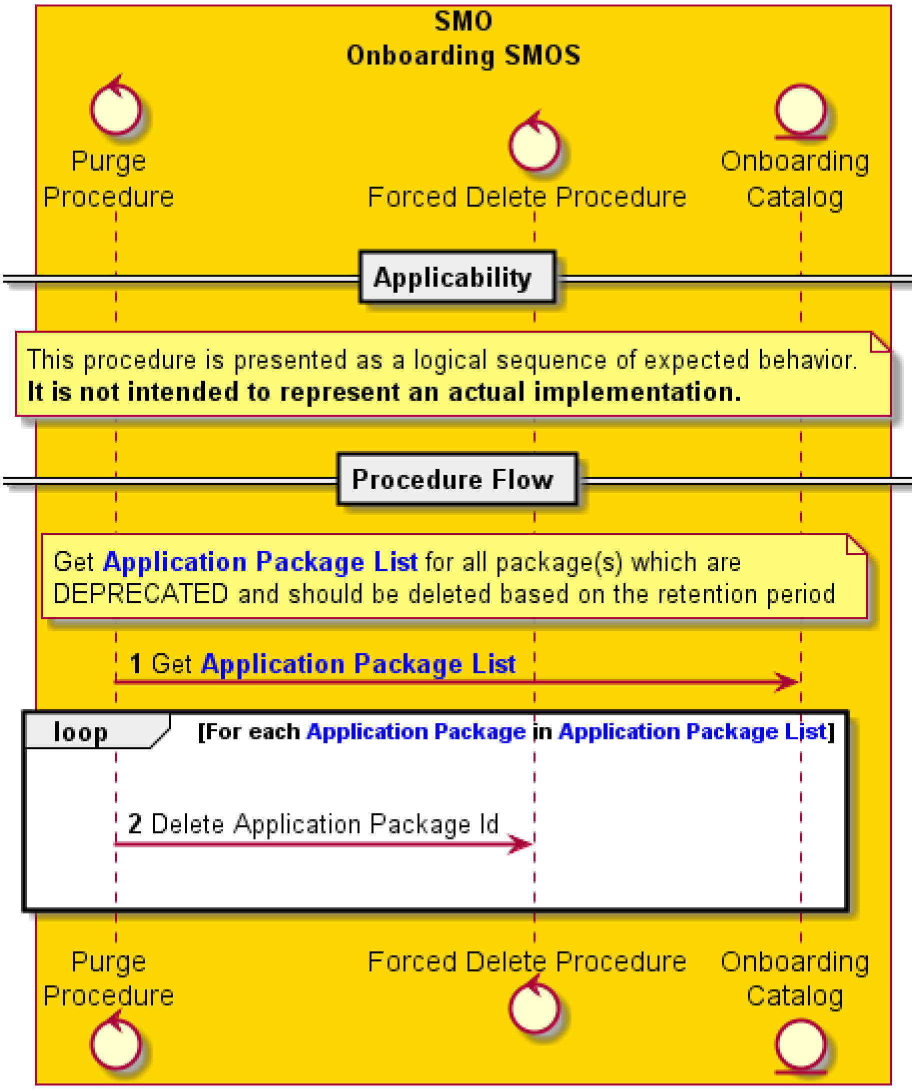

Figure 7.2.12.2-1 Purge procedure interaction diagram

8 Onboarding service requirements

8.1 Requirement conventions

Requirement categories are described in the “Management interface specification methodology” [i.6], clause A.1.3.The Onboarding Service will have functional (FUN), non-functional (NON), and administrative (ADM) requirement categories.

Requirements are to be written based on the following template:

REQ-Label-Category-Number

where "Label" is an abbreviation for the application type that the requirement is scoped to, category is the abbreviation listed above, and “Number” is a unique 2-digit numberical value within the label-category.

- ONBD Requirements specific to the Onboarding procesee
- CTLG Requirements specific to Catalog Exposure
- AFA Requirements specific to Artifact Access
- CNFG Requirements specific to configuration of the Onboarding Service

8.2 Onboarding requirements

REQ-ONBD-FUN-01: The Onboarding Service shall provide an interface to initiate the onboarding of an Application Package that is initially not trusted.

REQ-ONBD-FUN-02: The Onboarding Service shall verify that a signature for the expected vendor has been properly provided, as specified by O-RAN Security Requirements [1], for the package and/or artifacts.

REQ-ONBD-FUN-03: The Onboarding Service shall validate the integrity of artifacts contained in the Application Package as specified by O-RAN Security Requirements [1].

REQ-ONBD-FUN-04: The Onboarding Service shall unpack all artifacts from the Application Package.

REQ-ONBD-FUN-05: The Onboarding Service shall notify interested downstream consumers of events which change the state of an onboarding package.

8.3 Catalog requirements

REQ-CTLG-FUN-01: The Onboarding Service shall provide a query service to enable access to the Application Package as received from the vendor.

REQ-CTLG-FUN-02: The Onboarding Service shall provide a query service to access the metadata contained in the Application Package.

REQ-CTLG-FUN-03: The Onboarding Service shall provide a query service to retrieve artifact details for artifacts contained in the Application Package.

8.4 Artifact access requirements

REQ-AFA-FUN-01: The Onboarding Service shall provide the exposure of File System services enabling service consumers to be able to open and read artifacts extracted from an Application Package.

REQ-AFA-FUN-02: The Onboarding Service shall provide the ability to identify an extracted artifact from its path, relative or absolute, as identified in a reference used in another artifact.

8.5 Service configuration requirements

REQ-CNFG-ADM-01: The Onboarding Service shall provide the capability to change the amount of time that a package stays in the deprecated state before it is deleted.

REQ-CNFG-ADM-02: The Onboarding Service shall provide the capability to modify the service configuration only to a user with specific administrative privileges.

REQ-CNFG-ADM-03: The Onboarding Service shall provide the capability for any user to fetch the current service configuration parameters.

9 Onboarding service Information Model

Not specified in this version of the specification.

Annex A (informative): SMO platform assumptions

A.1 EventBus services

The Onboarding SMOS assumes that there is an Event Messaging Bus provided by the SMO Platform. It is expected that this service provides an anonymous publish/subscribe model. Such that the publisher(s) and subscribers(s) need not be directly aware of each other. The service also assumes that the Push-Push model is supported so that the service does not need to poll the bus for new events. Instead, an identified callback procedure in the topic subscription request is invoked whenever a subscribed message is received. The service also assumes that topic management is abstracted, although this might be by another service such as the Data Management Exposure (DME) SMOS as described in the Decoupled SMO Architecture [i.3].

A.2 Service Management Exposure SMOS

The Onboarding SMOS assumes that the Service Management and Exposure (SME) SMOS provides the ability for the Onboarding SMOS to register its services and discover the needed services from the SMO Platform. The Onboarding SMOS does not have any other dependencies on other SMOS.

A.3 Data Management Exposure SMOS

The Onboarding SMOS assumes that the Data Management and Exposure (DME) SMOS provides the ability for the Onboarding SMOS to register its data products to invoke procedures via a message over the Event Bus. Additionally, it will register data events that it will publish when an Application Package is onboarded, deprecated, or deleted.

A.4 FileSystem-As-A-Service

The Onboarding SMOS assumes that physical storage management plan of the SMO is abstracted from services, like the Onboarding SMOS, which need to store files. Therefore, it is expected that the SMO platform provides a service for this. The Onboarding service is not aware of where the files are physically stored other than what is exposed by the FileSystem-as-a-Service interface.

A.5 Database-As-A-Service

Like the FileSystem As-A-Service the Onboarding SMOS is expected to also maintain metadata about an onboarded package including the links to the physical files managed by the FileSystem-As-A-Service. The exact implementation of the structured data storage may depend on the robustness of services offered by the SMO platform. Structured storages could vary from key-value stores, object stores, and/or relational databases. The basic capabilities of Database-As-A-Service needs to support the ability to define a logical area for the "database", the structure for the "tables", the ability to insert, query, update, and delete data from the "tables".

A.6 Logging service

It is assumed that the SMO Platform will have a common logging management service. This service will abstract how and where log entries are stored. It will also provide a common log format such that logs from multiple services can be analyzed when troubleshooting a problem.

Annex (informative): Change history/Change request (history)

<Change history/Change request (history) is mandatory>
<Please delete the sample entries below and replace with entries for this document>

<Note that if this document will be submitted to ETSI for publication, ETSI may insert an additional "History" table on the following page. This should not be created by the authors of the current document.>

Date	Revision	Description
2024.06.24	00.00.01	Create initial O-RAN version, from TS Template
2024.11.12	00.00.02	ATT-2024.07.08-WG10-CR-0001-OAM APP LCM Content-v05 ATT-2024.09.16-WG10-CR-0003-Scope Correction-v04
2024.11.12	00.00.03	ATT-2024.09.16-WG10-CR-0004-Onboarding Use Cases-v02
2024.11.12	00.00.04	General Aspects: ATT-2024.09.16-WG10-CR-0005-GA ETSI Alignment-v04 ATT-2024.09.16-WG10-CR-0007-GA Distributed SMOS-v03 ATT-2024.09.16-WG10-CR-0008-GA Notifications-v02 ATT-2024.09.16-WG10-CR-0010-GA Reference vs Copy-v02 ATT-2024.09.18-WG10-CR-0011-GA Procedure Idempotence-v02 ATT-2024.09.18-WG10-CR-0012-GA Role Based Procedures-v02
2024.11.12	00.00.05	Principles: ATT-2024.09.18-WG10-CR-0013-Principle of Encapsulations-v04 Editorial Changes for publication
2024.12.02	01.00	Initial Published version
2025.04.30	01.00.05	CRs: ATT-2025.03.21-WG10-CR-0029-Outline Correction for Use Cases and Procedures-v01 ATT-2024.09.19-WG10-CR-0015-Procedure Actors-v04 ATT-2024.09.19-WG10-CR-0016-Procedure Exception Handling-v05 ATT-2024.09.16-WG10-CR-0006-GA SMO Platform-v02 ATT-2024.09.16-WG10-CR-0009-GA Deprecation vs Deletion-v04 ERI-2024.11.19-WG10-CR-0143-OnboardSMOSGAP-hierarchical-v01 ATT-2025.03.18-WG10-CR-0027-API Handling-v02 ATT-2024.09.19-WG10-CR-0026-Procedure Configuration Query-v04 ATT-2024.09.19-WG10-CR-0025-Procedure Configuration Update-v05 ATT-2024.09.19-WG10-CR-0021-Procedure Deprecate Package-v07
2025.05.05	01.00.06	CRS: ATT-2024.09.18-WG10-CR-0014-Principles on Artifact Handling-v06 ATT-2024.09.19-WG10-CR-0017-Procedure Onboard-v06 ATT-2024.09.19-WG10-CR-0018-Procedure Verify-v08 ATT-2024.09.19-WG10-CR-0019-Procedure Unpack-v07 ERI-2025.04.01-WG10-CR-0165-OnboardingAltFlow_v03 ATT-2025.03.19-WG10-CR-0028-Procedure Actor Update-v01 ATT-2024.09.19-WG10-CR-0020-Procedure Query-v06 ATT-2025.04.02-WG10-CR-0031-Procedure File Access-v01 ATT-2024.09.19-WG10-CR-0022-Procedure Forced Delete-v07 ATT-2025.03.21-WG10-CR-0030-Initial Service Requirements-v02
2025.07.01	01.00.07	CRS: ATT-2024.09.19-WG10-CR-0023-Procedure Cancel Deprecation-v06 ATT-2024.09.19-WG10-CR-0024-Procedure Purge-v06
2025.07.14	02.00	July 2025 Release version 02.00

